

# 顺路旅行规划 Agent——课程大作业报告

姓名：孟奕晨 学号：2025302912

2026-06-22

## 目录

|                       |          |
|-----------------------|----------|
| <b>1 摘要</b>           | <b>3</b> |
| <b>2 选题背景与设计思想</b>    | <b>3</b> |
| 2.1 问题背景              | 3        |
| 2.2 现有方案的不足           | 3        |
| 2.2.1 静态攻略的信息适配度低     | 3        |
| 2.2.2 表单式推荐缺乏自然交互     | 3        |
| 2.2.3 单次 LLM 生成缺少可靠边界 | 3        |
| 2.3 项目目标              | 4        |
| 2.4 项目价值              | 4        |
| 2.5 设计原则              | 4        |
| 2.6 技术路线              | 4        |
| <b>3 Specs 规格文档</b>   | <b>4</b> |
| 3.1 Product Spec      | 4        |
| 3.1.1 用户角色与使用场景       | 4        |
| 3.1.2 功能需求            | 5        |
| 3.1.3 非功能需求           | 5        |
| 3.1.4 业务规则            | 5        |
| 3.1.5 MVP 边界          | 5        |
| 3.2 Architecture Spec | 6        |
| 3.2.1 分层规格            | 6        |
| 3.2.2 状态规格            | 6        |
| 3.2.3 错误与降级规格         | 6        |
| 3.3 API Spec          | 6        |
| 3.3.1 POST /api/chat  | 6        |
| 3.3.2 GET /api/health | 7        |
| <b>4 系统架构与设计</b>      | <b>7</b> |
| 4.1 总体架构              | 7        |
| 4.2 Agent 交互时序        | 7        |
| 4.3 数据流设计             | 7        |
| 4.4 约束抽取设计            | 9        |
| 4.5 工具设计              | 9        |
| 4.5.1 地理编码工具          | 9        |
| 4.5.2 天气工具            | 9        |
| 4.5.3 工具不足与事实边界       | 9        |
| 4.6 模型调用与流式处理         | 9        |
| 4.7 预算算法              | 9        |

|          |                   |           |
|----------|-------------------|-----------|
| 4.8      | 安全与隐私设计           | 10        |
| <b>5</b> | <b>关键实现与代码展示</b>  | <b>10</b> |
| 5.1      | 核心 Agent 循环       | 10        |
| 5.2      | 旅行画像与缺失判断         | 10        |
| 5.3      | 中文约束解析            | 11        |
| 5.4      | 工具调用与容错           | 11        |
| 5.5      | LLM 流式响应          | 11        |
| 5.6      | Web 交互            | 11        |
| 5.7      | 配置文件              | 11        |
| 5.8      | AI IDE 协作过程       | 11        |
| <b>6</b> | <b>测试与评估</b>      | <b>12</b> |
| 6.1      | 测试策略              | 12        |
| 6.2      | 已实现自动化测试          | 12        |
| 6.3      | 已进行接口验证           | 12        |
| 6.4      | 缺陷发现与修复记录         | 12        |
| 6.5      | 评估指标设计            | 13        |
| 6.6      | 当前局限              | 13        |
| <b>7</b> | <b>系统升级与扩展</b>    | <b>13</b> |
| 7.1      | 近期迭代：提高规划可靠性      | 13        |
| 7.1.1    | 增加 POI 与路线工具      | 13        |
| 7.1.2    | 增加约束验证 Agent      | 13        |
| 7.1.3    | 结构化行程输出           | 13        |
| 7.2      | 中期迭代：工程化与生产可用性    | 14        |
| 7.3      | 长期演进：动态旅行助手       | 14        |
| 7.4      | AI 能力演进路径         | 14        |
| <b>8</b> | <b>课程总结</b>       | <b>14</b> |
| 8.1      | 工程认知转变            | 14        |
| 8.2      | 开发过程反思            | 14        |
| 8.3      | AI IDE 使用体会       | 14        |
| 8.4      | 对课程的建议            | 15        |
| <b>9</b> | <b>附录 A：运行与复现</b> | <b>15</b> |
| 9.1      | 环境安装              | 15        |
| 9.2      | 配置模型              | 15        |
| 9.3      | 启动系统              | 15        |
| 9.4      | 执行测试              | 15        |

| 字段        | 内容                    |
|-----------|-----------------------|
| 课程名称      | 企业级应用软件设计与开发          |
| 项目名称      | 顺路旅行规划 Agent          |
| 方向        | 方向二：企业级应用软件的 Agent 改造 |
| 指导教师      | 戚欣                    |
| 学生姓名 / 学号 | 孟奕晨 / 2025302912      |
| 专业        | 计算机技术                 |
| 提交日期      | 2026-06-22            |

# 1 摘要

传统旅行推荐系统通常依赖用户一次性填写表单，或直接通过大语言模型生成一段行程文本。前者交互成本较高，后者虽然自然，但容易在约束缺失时直接作答，并出现预算不守恒、路线折返、每日安排过密、实时信息失真等问题。本项目设计并实现了“顺路旅行规划 Agent”：用户通过网页对话描述目的地、人数、天数、预算和偏好，系统持续维护结构化旅行画像；当信息不完整时主动追问，当信息完整时调用地点和天气工具，并将用户约束及工具上下文交给大语言模型生成详细行程。模型未配置、网络异常或请求超时，系统自动切换到本地规则规划器，保证核心功能仍然可用。

系统采用 Python、FastAPI、Pydantic、HTTPX 和原生 Web 技术实现，支持 OpenAI Chat Completions 兼容接口，API Key 仅通过环境变量配置。当前 MVP 已实现多轮约束收集、中文人数及预算解析、实时地点与天气查询、流式模型响应、本地降级、预算分类和 Markdown 行程呈现，并通过 5 项自动化测试。项目验证了 Agent 工程并非简单“调用一次模型”，而是由状态、工具、约束、错误处理、降级策略和评估机制共同组成的软件闭环。

**关键词：**Agentic AI；旅行规划；多轮对话；工具调用；结构化状态；大语言模型；失败降级

## 2 选题背景与设计思想

### 2.1 问题背景

旅行规划是一项典型的多约束决策任务。用户通常只会先给出模糊目标，例如“下周去广州附近玩十天，要吃好住好”，但一份真正可执行的计划至少还需要处理以下信息：

- 硬约束：目的地、旅行日期、旅行天数、人数、总预算；
- 软偏好：自然、人文、艺术、美食、亲子、购物、住宿品质；
- 时空约束：景点地理位置、营业日期、游览耗时和交通时间；
- 动态条件：天气、临时闭馆、预约规则、票价变化；
- 风险约束：体力消耗、排队时间、预算机动空间和消费陷阱。

这类任务不能只依赖关键词推荐。系统需要先识别用户意图，再判断信息是否充分，必要时继续追问，之后调用外部工具获得事实数据，最后在预算和时间限制下组织输出。因此，旅行规划天然适合用 Agent 架构改造。

### 2.2 现有方案的不足

#### 2.2.1 静态攻略的信息适配度低

传统攻略内容面向大众，通常按照“必去榜单”组织，无法准确适配不同人数、预算和兴趣。用户还需要手工判断景点是否相邻、是否需要预约、一天能否游览完成，信息检索成本较高。

#### 2.2.2 表单式推荐缺乏自然交互

结构化表单便于系统处理，却要求用户一次性理解并填写所有字段。实际用户往往只知道大概想法，后续要求是在交流过程中逐步明确的，因此一次性表单不符合真实决策过程。

#### 2.2.3 单次 LLM 生成缺少可靠边界

直接把用户原话发送给大模型虽然能生成流畅文本，但存在四类突出风险：

1. 在天数、预算等字段缺失时擅自假设；
2. 输出真实感很强但未经核验的票价和营业时间；
3. 景点堆叠过多或跨区域折返；
4. 模型或网络失败后整个功能不可用。

## 2.3 项目目标

本项目的目标不是代替地图、票务和预订平台，而是构建一个可扩展的旅行规划 Agent MVP，完成从自然语言需求到结构化、可核算行程的转换。具体目标如下：

- 通过多轮对话收集目的地、天数、预算和偏好；
- 在核心信息不足时只追问缺失项，不生成伪精确方案；
- 每天控制在 2 个核心景点和 1 个可选活动，最多不超过 3—4 个景点；
- 明确给出时间节点、交通方式、耗时、餐饮和住宿建议；
- 输出住宿、餐饮、交通、门票和机动金预算，并保证总额守恒；
- 调用公开地点和天气服务，为模型提供可验证上下文；
- 支持 OpenAI 兼容模型，并在模型失败时自动降级；
- 不在代码或公开模板中硬编码 API Key。

## 2.4 项目价值

从用户价值看，系统降低了收集攻略、组合路线和估算预算的时间成本。从工程价值看，本项目展示了如何把一个普通 Web 推荐功能改造成 Agent：系统不再只是执行固定流程，而是根据当前状态决定“继续追问、调用工具、调用模型或降级”。此外，结构化画像、工具边界和自动化测试使生成式功能具备了传统软件所需的可解释性与可验证性。

## 2.5 设计原则

项目遵循以下原则：

- **先确认，后生成**：核心字段缺失时停止规划并定向追问；
- **约束优先**：预算、天数和人数优先级高于景点丰富度；
- **地理聚类**：同一天尽量安排相邻区域，减少折返；
- **节奏合理**：为交通、用餐、排队和休息保留时间；
- **事实有来源**：实时信息以工具结果为准，未知内容明确提示复核；
- **失败可降级**：模型、地图或天气服务失败不导致系统完全不可用；
- **密钥不入库**：敏感配置仅由本地 .env 环境文件注入。

## 2.6 技术路线

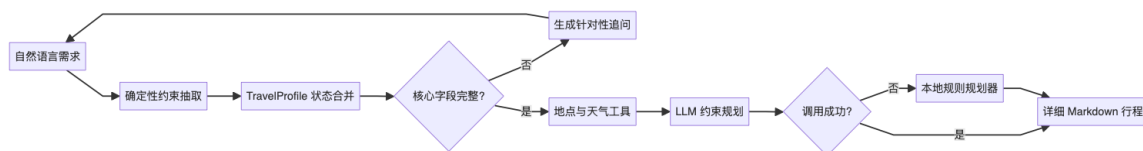


Figure 1: 技术路线

# 3 Specs 规格文档

## 3.1 Product Spec

### 3.1.1 用户角色与使用场景

主要用户是希望快速获得初步旅行方案的个人或小组。典型场景包括：

- 用户只提供目的地，系统继续询问天数、预算和兴趣；
- 用户一次给出完整需求，系统直接生成行程；
- 用户用“两个人”“预算 2 万”等中文自然表达，系统仍能抽取约束；
- 模型配置正确，系统生成具体景点方案；
- 模型超时或未配置，系统返回明确标注的离线骨架而不是报错白屏。

3.1.2 功能需求

| 编号    | 功能       | 优先级 | 验收标准                                        |
|-------|----------|-----|---------------------------------------------|
| FR-01 | 接收网页对话输入 | P0  | 用户可发送 1—2000 字消息并看到回复                       |
| FR-02 | 抽取旅行核心信息 | P0  | 能识别目的地、天数、预算、人数和偏好                          |
| FR-03 | 多轮状态合并   | P0  | 后一轮补充字段后保留前一轮已知信息                           |
| FR-04 | 缺失字段追问   | P0  | 缺少核心字段时状态为 <code>collecting</code> ，且只询问缺失项 |
| FR-05 | 地点与天气查询  | P1  | 工具成功时返回地点坐标和近期天气上下文                         |
| FR-06 | LLM 行程生成 | P0  | 字段完整且模型可用时输出多日详细行程                          |
| FR-07 | 离线降级     | P0  | LLM 未配置、超时或异常时返回规则行程骨架                      |
| FR-08 | 预算核算     | P0  | 输出分类预算，分类合计等于用户总预算                          |
| FR-09 | 加载反馈     | P1  | 请求期间页面显示“正在生成中...”                          |
| FR-10 | 健康检查     | P1  | <code>/api/health</code> 返回服务状态及规划器模式       |

3.1.3 非功能需求

| 类别   | 规格                                                            |
|------|---------------------------------------------------------------|
| 安全性  | API Key 不写入源码， <code>.env</code> 被 <code>.gitignore</code> 排除 |
| 可用性  | 外部工具或模型失败时能够降级，不返回服务器堆栈给用户                                    |
| 可维护性 | API、模型、工具、数据模型和前端分模块组织                                        |
| 可测试性 | 约束抽取、追问和预算逻辑可脱离外部网络进行单元测试                                     |
| 兼容性  | 支持 Python 3.9+ 和 OpenAI Chat Completions 兼容接口                 |
| 性能   | 普通本地追问应快速返回；LLM 连续 30 秒无数据时超时降级                               |
| 可解释性 | 离线输出明确标注为规则骨架；实时票价和营业状态提示复核                                   |

3.1.4 业务规则

1. `destination`、`days`、`budget`、`preferences` 是生成完整方案前的必需字段；
2. 人数默认为 1，允许范围为 1—20 人；
3. 旅行天数允许范围为 1—30 天；
4. 每日核心景点数量控制在 2 个左右，并允许 1 个可选夜间活动；
5. 本地预算比例为住宿 35%、餐饮 25%、交通 20%、门票体验 15%、机动金 5%；
6. 当人均日预算低于约 350 元时，离线规划器追加预算偏紧提示；
7. 模型不能把工具上下文中不存在的实时价格或营业状态描述成已核实事实。

3.1.5 MVP 边界

当前版本不执行机票、酒店和门票下单；不提供导航级路线计算；不保证第三方公开数据绝对实时；不持久保存用户账号和身份信息；不对模型生成内容作法律、医疗或安全担保。系统输出定位为“规划建议”，出发前仍需通过官方渠道复核。

## 3.2 Architecture Spec

### 3.2.1 分层规格

| 层级        | 模块                 | 主要职责                            |
|-----------|--------------------|---------------------------------|
| 表现层       | src/web/index.html | 对话展示、发送请求、加载提示、保存会话 ID          |
| API 层     | src/main.py        | 路由、请求校验、会话 ID 创建、响应序列化          |
| 领域模型层     | src/models.py      | 定义旅行画像、请求与响应 Schema             |
| Agent 编排层 | src/planner.py     | 状态合并、缺失判断、工具调用、模型调用和降级          |
| 工具层       | src/tools.py       | Nominatim 地理编码和 Open-Meteo 天气查询 |
| 配置层       | src/config.py      | 从环境变量读取模型 URL、Model、Key 和超时     |

### 3.2.2 状态规格

核心状态对象为 TravelProfile:

| 字段          | 类型           | 说明                 |
|-------------|--------------|--------------------|
| destination | str   None   | 城市或区域，例如“广州及其附近”   |
| days        | int   None   | 旅行天数，范围 1—30       |
| budget      | float   None | 全体旅行者总预算           |
| travelers   | int          | 人数，默认 1，范围 1—20    |
| preferences | list[str]    | 自然、人文、艺术、美食、亲子、购物等 |
| departure   | str   None   | 出发地，当前保留扩展         |
| start_date  | date   None  | 明确日期，当前支持数字日期解析    |

服务端使用 dict[session\_id, TravelProfile] 保存会话画像。该设计简单、适合实验环境，但应用重启后状态会丢失，也不支持多进程共享。生产版本应迁移到 Redis 或 MySQL。

### 3.2.3 错误与降级规格

| 故障            | 当前处理               |
|---------------|--------------------|
| 地理编码无结果       | 返回空工具上下文，继续规划      |
| 地点/天气网络异常     | 捕获 HTTP 异常，不中断主流程  |
| API Key 未配置   | 跳过模型，直接进入本地规则规划器   |
| 模型 HTTP 错误    | 记录状态码与截断响应，进入本地规划器 |
| 模型连续 30 秒无流数据 | 记录超时日志，进入本地规划器     |
| 模型流事件格式异常     | 忽略单个异常事件，继续读取后续内容  |
| 模型返回空内容       | 记录警告并降级            |

## 3.3 API Spec

### 3.3.1 POST /api/chat

用途：提交一轮用户消息，创建或继续旅行规划会话。

请求示例：

```
{
  "message": " 两个人去杭州玩 3 天, 预算 5000 元, 喜欢自然和人文",
  "session_id": null
}
```

字段说明:

| 字段         | 必需 | 约束       | 说明        |
|------------|----|----------|-----------|
| message    | 是  | 1—2000 字 | 当前用户消息    |
| session_id | 否  | UUID 字符串 | 空值时由服务端生成 |

完整响应示例:

```
{
  "session_id": "3e84c0a8-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "reply": "## 杭州 3 日旅行方案……",
  "status": "complete",
  "profile": {
    "destination": " 杭州",
    "days": 3,
    "budget": 5000.0,
    "travelers": 2,
    "preferences": [" 人文", " 自然"],
    "departure": null,
    "start_date": null
  },
  "sources": ["OpenStreetMap Nominatim", "Open-Meteo"]
}
```

status=collecting 表示仍需补充信息, status=complete 表示已经进入规划阶段。complete 不等于所有实时事实均已核验, 调用方仍应查看回复中的复核提示。

### 3.3.2 GET /api/health

用途: 检查 Web 服务和规划器配置模式。

```
{"status": "ok", "planner": "llm"}
```

其中 planner=llm 仅表示检测到非空 API Key, 不代表上游模型服务必然可用。后续版本可扩展为主动探测和依赖健康状态。

## 4 系统架构与设计

### 4.1 总体架构

架构的核心不是模型本身, 而是 PlannerAgent 的决策能力。它根据旅行画像当前状态决定下一步动作, 形成“感知输入—更新状态—选择动作—输出结果”的简化 Agent 循环。

### 4.2 Agent 交互时序

### 4.3 数据流设计

输入首先经过确定性解析, 不直接进入大模型。update\_profile 使用正则表达式和关键词字典识别字段, 再与旧画像合并。字段完整后, 工具层返回结构化 JSON; Agent 将画像和工具结果序列化后发送给模型。这样可以清楚

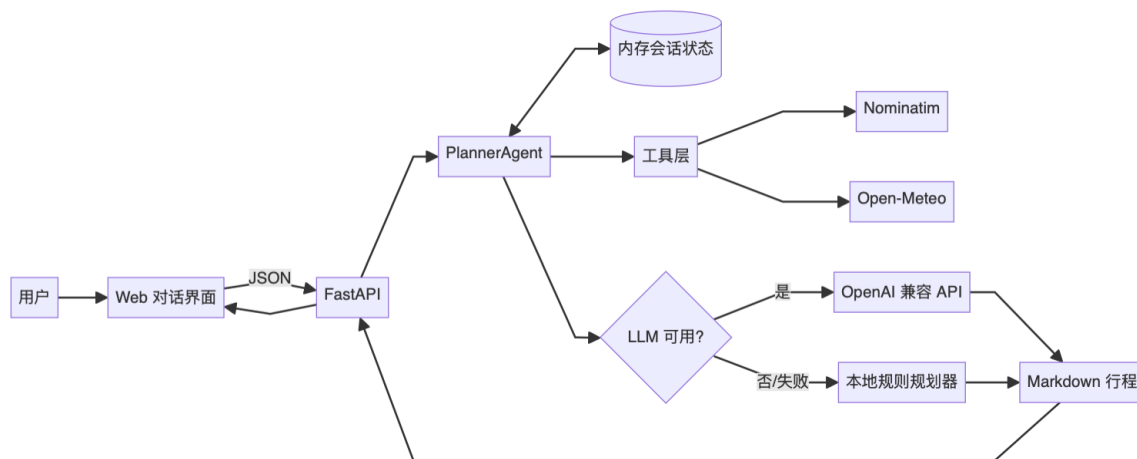


Figure 2: 系统总体架构



Figure 3: Agent 交互时序

区分三种数据来源：

1. 用户明确表达的需求；
2. 外部工具返回的事实；
3. 模型生成的建议。

该边界可以降低提示词歧义，也便于后续增加审计、缓存和事实引用。

## 4.4 约束抽取设计

当前 MVP 采用确定性解析而不是额外调用模型抽取字段，原因是目的地、天数、人数和预算的格式相对稳定，规则方法成本低、响应快、容易测试。例如：

- 预算 20000、预算 2 万、5000 元 均可归一化为数值预算；
- 2 人、两个人、十二人 可统一转换为整数人数；
- “自然风光”“博物馆”“当地美食”等关键词映射为标准偏好标签；
- 多轮对话使用旧画像作为基础，只覆盖本轮新识别出的字段。

规则方法的局限是无法覆盖所有口语和复杂否定，例如“不喜欢购物”“大人两位加一个儿童”。后续可采用“规则优先 + LLM 结构化抽取兜底”的混合策略。

## 4.5 工具设计

### 4.5.1 地理编码工具

系统通过 OpenStreetMap Nominatim 把目的地名称解析为规范地址、纬度和经度。请求设置专用 User-Agent，并限制只返回一个候选结果。若没有匹配结果，则返回空字典。

### 4.5.2 天气工具

获得坐标后，系统调用 Open-Meteo 获取未来 7 天的天气代码、最高温、最低温和最高降雨概率。该服务不需要 API Key，适合课程项目演示。天气只在预测窗口内有意义，因此系统不会把短期预报用于很久以后的日期。

### 4.5.3 工具不足与事实边界

当前工具尚未查询具体景点 POI、景点间路线矩阵、营业时间和官方票价。因此，模型能够给出具体景点建议，但不能宣称所有实时信息已经核验。系统提示词明确要求：工具上下文没有提供的实时价格和营业状态必须标注待复核。这是当前版本最重要的能力边界。

## 4.6 模型调用与流式处理

模型层采用 Chat Completions 兼容协议，可以通过 .env 配置 URL、模型名和 Key。系统提示词规定每日景点上限、交通信息、预算表和防坑提示。temperature=0.3 用于降低随机性，max\_tokens=6000 用于容纳多日方案。

最初版本采用非流式请求，十日行程在 30 秒内无法生成完整响应时会超时，但上游平台仍可能记录已生成 Token。改进后系统使用 SSE 流式接收：只要模型持续输出数据，连接就不会因为等待整篇结果而超时；连续 30 秒没有任何数据才触发降级。这一修复减少了“API 已计费但页面只显示离线骨架”的问题。

## 4.7 预算算法

离线规则引擎使用以下预算拆分：

$$[ B = B_{\{hotel\}} + B_{\{food\}} + B_{\{transport\}} + B_{\{ticket\}} + B_{\{reserve\}} ]$$

其中：

$$[ B_{\{hotel\}}=0.35B, \quad B_{\{food\}}=0.25B, \quad B_{\{transport\}}=0.20B ]$$

[ B\_{ticket}=0.15B, B\_{reserve}=B-B\_{hotel}-B\_{food}-B\_{transport}-B\_{ticket} ]

最后一项通过总预算减去其他四项计算，从而吸收四舍五入误差，保证预算总额守恒。模型模式下则要求模型按用户预算输出分类表，但后续仍应增加机器可执行的二次预算校验。

## 4.8 安全与隐私设计

- .env 存放真实密钥并被 Git 忽略；
- .env.example 只保存空白模板，禁止出现真实 Key；
- 浏览器不接触模型 Key，所有模型请求都由后端发起；
- 日志只记录错误类型和截断响应，不主动输出 Authorization Header；
- 当前系统不要求登录，也不采集身份证、手机号等身份信息；
- 会话画像暂存在内存中，服务重启后清除。

## 5 关键实现与代码展示

### 5.1 核心 Agent 循环

PlannerAgent.respond 是系统的核心编排入口，其逻辑可以概括为：

```
async def respond(self, session_id, message):
    profile = update_profile(self.profiles.get(session_id, TravelProfile()), message)
    self.profiles[session_id] = profile

    if profile.missing:
        return clarification(profile), "collecting", profile, []

    context, sources = await enrich_destination(profile.destination)
    plan = await llm_plan(profile, context) or local_plan(profile, context)
    return plan, "complete", profile, sources
```

这段循环体现了 Agent 的动作选择：字段不完整时选择“追问”；字段完整时选择“调用工具”；工具返回后选择“调用模型”；模型失败时选择“执行本地规划”。

### 5.2 旅行画像与缺失判断

Pydantic 模型同时承担运行时数据校验和 API Schema 生成。missing 属性集中定义核心字段完整性，避免前端、API 和模型层分别维护不同判断标准。

```
@property
def missing(self) -> list[str]:
    fields = []
    if not self.destination:
        fields.append(" 目的地")
    if not self.days:
        fields.append(" 旅行天数")
    if not self.budget:
        fields.append(" 总预算")
    if not self.preferences:
        fields.append(" 兴趣偏好")
    return fields
```

## 5.3 中文约束解析

系统对常见中文数字进行了归一化，例如“一”“二”“两”“十二”。预算解析允许带单位和不带单位两种表达，其中无单位金额必须出现在“预算”等明确关键词之后，以降低误把日期或人数识别为预算的风险。

在开发过程中，自动化测试曾发现“两个人”没有被识别，以及“预算 20000”因缺少“元”而被漏掉。两个问题均通过增加解析规则和回归测试解决。这体现了测试对自然语言边界条件的重要作用。

## 5.4 工具调用与容错

`enrich_destination` 将地理编码和天气查询封装为单一工具入口。任何网络错误、响应格式错误或地点不存在都被转换为空上下文，使 Agent 能继续执行降级路径。来源列表只在工具真实返回数据后添加，避免产生虚假的来源声明。

## 5.5 LLM 流式响应

模型请求启用 `stream=True`，后端逐行读取 SSE 事件并拼接 `delta.content`。DeepSeek 等推理模型可能先输出 `reasoning_content`，系统只收集最终 `content`，但中间流事件仍会刷新网络读取超时。异常事件只影响单个数据片段，不会使整个请求立即失败。

## 5.6 Web 交互

前端不依赖大型框架，使用 `fetch` 调用 `/api/chat`。会话 ID 保存在浏览器 `localStorage` 中，用户发送消息后立即插入“正在生成中...”气泡，并禁用发送按钮；请求完成后在原位置替换为行程或错误提示。该实现减少了长行程生成期间用户误以为页面无响应的问题。

## 5.7 配置文件

真实配置写入项目根目录 `.env`：

```
LLM_API_KEY=  
LLM_BASE_URL=https://api.openai.com/v1  
LLM_MODEL=gpt-4.1-mini  
LLM_TIMEOUT=30
```

程序通过 `pydantic-settings` 读取配置。如果 `LLM_API_KEY` 为空，健康检查返回 `planner=local`，Agent 不会尝试发送模型请求。

## 5.8 AI IDE 协作过程

本项目使用 Codex 辅助完成需求拆解、目录设计、代码生成、故障定位、测试补充和文档整理。有效的人机协作方式不是一次性要求 AI 生成全部代码，而是把任务拆成“建立 MVP—执行测试—复现问题—补充回归测试—完善文档”的迭代过程。

正式提交时建议插入以下截图，并为每张图添加编号和说明：

1. 图 4-1: AI IDE 中读取原始需求并拆分模块；
2. 图 4-2: PlannerAgent 核心循环代码；
3. 图 4-3: 信息不完整时的定向追问；
4. 图 4-4: 网页“正在生成中...”状态；
5. 图 4-5: 完整多日行程与预算表；
6. 图 4-6: FastAPI 自动接口文档；
7. 图 4-7: pytest 全部通过的终端输出。

6 测试与评估

6.1 测试策略

测试分为四层：

- 单元测试：验证字段解析、缺失判断和预算算法；
- API 测试：验证健康检查、请求校验和完整对话响应；
- 集成测试：验证地点、天气和模型接口；
- 人工行为评估：检查路线合理性、个性化程度和防坑信息。

外部 API 具有网络波动和数据变化，不适合作为所有单元测试的前置条件。因此，确定性业务规则优先使用离线测试，外部集成另行验证。

6.2 已实现自动化测试

当前测试文件包含 5 项测试：

| 测试                                   | 验证内容                    | 结果 |
|--------------------------------------|-------------------------|----|
| test_extracts_core_constraint        | 目的地、3 天、5000 元、两人、自然和人文 | 通过 |
| test_requests_missing_information    | 只提供成都时继续询问天数、预算和偏好      | 通过 |
| test_budget_is_preserved             | 3 日方案数量正确且预算总额为 5000 元  | 通过 |
| test_extracts_chinese_traveler_count | “12”解析为 12              | 通过 |
| test_accepts_budget_without_currency | 多轮补充“预算 20000”后图像完整     | 通过 |

最近一次执行结果：

```
..... [100%]
5 passed
```

6.3 已进行接口验证

使用 FastAPI TestClient 验证了以下路径：

- GET /api/health 返回 HTTP 200；
- 完整杭州需求返回 HTTP 200 和 status=complete；
- 返回画像中的目的地为杭州、天数为 3、人数为 2、预算为 5000；
- 返回行程包含预算总额。

此外，开发环境中已成功启动 Uvicorn，主页和两次 /api/chat 请求均返回 HTTP 200。浏览器请求 /favicon.ico 的 404 只表示未配置站点图标，不影响业务功能。

6.4 缺陷发现与修复记录

| 缺陷                    | 根因                  | 修复                 |
|-----------------------|---------------------|--------------------|
| “两个人”被识别为默认 1 人       | 人数正则只支持阿拉伯数字        | 增加中文小数字解析与测试       |
| “预算 20000”被反复追问       | 原规则强制要求“元/块/千/万”    | 预算关键词后允许省略单位，默认元   |
| API 有 Token 记录但输出离线骨架 | 非流式长响应超过 30 秒，异常被降级 | 改用 SSE 流式读取并增加错误日志 |
| 用户等待时缺少反馈             | 前端只在最终响应后插入消息       | 增加动态“正在生成中...”气泡   |

| 缺陷              | 根因         | 修复                                                        |
|-----------------|------------|-----------------------------------------------------------|
| 示例文件曾可能包含真实 Key | 错误地编辑了公开模板 | 重建空白 <code>.env.example</code> 并保留 <code>.env</code> 忽略规则 |

## 6.5 评估指标设计

Final 阶段建议建立至少 30 条 Benchmark，覆盖单人/多人、短途/长途、低预算/高预算、中文数字、缺失信息、冲突约束和外部服务失败。核心指标如下：

| 指标       | 计算方式                 | 目标值         |
|----------|----------------------|-------------|
| 字段抽取准确率  | 正确字段数 / 标注字段总数       | $\geq 90\%$ |
| 缺失追问准确率  | 正确追问样例数 / 缺失样例总数     | $\geq 95\%$ |
| 预算违反率    | 超过用户预算的方案数 / 完整方案数   | 0%          |
| 单日景点超量率  | 超过 4 个核心景点的天数 / 总天数  | 0%          |
| 工具来源可追溯率 | 带有效来源的实时事实数 / 实时事实总数 | $\geq 90\%$ |
| 降级成功率    | 外部故障时返回可读骨架次数 / 故障次数 | 100%        |
| 首次追问延迟   | 本地追问的端到端耗时           | < 1 秒（建议目标） |

上述目标值是验收标准，不应在未执行完整 Benchmark 前写成实际达成率。当前能够确认的实测结果仅是 5 项自动化测试通过以及已执行的接口冒烟测试通过。

## 6.6 当前局限

1. 规则抽取未覆盖复杂否定、组合家庭结构和模糊预算；
2. “下周”等相对日期尚未归一化为明确日期；
3. 地点工具只解析目的地区域，没有检索完整 POI；
4. 未使用地图路线矩阵验证景点间真实通勤时间；
5. LLM 输出预算尚缺少独立程序化二次校验；
6. 会话保存在进程内存中，热重载会清空画像；
7. 前端目前按纯文本显示 Markdown，尚未进行富文本渲染；
8. 尚未实现用户认证、限流、缓存和完整可观测性。

## 7 系统升级与扩展

### 7.1 近期迭代：提高规划可靠性

#### 7.1.1 增加 POI 与路线工具

接入地图 POI 搜索、路线矩阵和公共交通接口，获得景点坐标、距离、通勤方式和预计耗时。规划前先对候选景点聚类，再让模型负责表达和个性化，可显著减少折返路线。

#### 7.1.2 增加约束验证 Agent

将生成过程拆为“规划 Agent + 审核 Agent”：规划 Agent 生成候选路线，审核 Agent 检查每日景点数、时间冲突、预算总和、开放时间和跨区通勤。审核失败时返回结构化问题并要求规划 Agent 修订，而不是直接把初稿展示给用户。

#### 7.1.3 结构化行程输出

要求模型输出符合 JSON Schema 的每日活动对象，再由后端渲染 Markdown。结构化输出可支持预算求和、地图展示、日历导出和单日重规划，也能避免仅靠字符串检查。

## 7.2 中期迭代：工程化与生产可用性

- 使用 Redis 保存会话状态，设置过期时间并支持多进程部署；
- 使用 MySQL 保存匿名化行程、用户反馈和评估样本；
- 增加请求 ID、结构化日志、耗时、Token 和降级原因指标；
- 对地理编码、天气和 POI 结果增加缓存，降低延迟和第三方压力；
- 增加接口限流、输入清洗、Prompt Injection 防护和内容安全策略；
- 使用 Docker 固化运行环境，并增加持续集成测试。

## 7.3 长期演进：动态旅行助手

系统可进一步支持旅行中的动态重规划。例如用户说“今天下雨”“某景点临时关闭”“上午起晚了”，Agent 根据剩余时间、当前位置、已购门票和用户体力重新安排当天路线。多人旅行场景还可引入偏好权重和冲突协商，形成兼顾团队成员的折中方案。

## 7.4 AI 能力演进路径

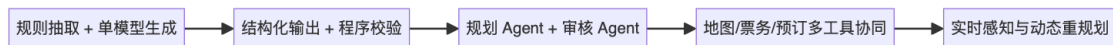


Figure 4: AI 能力演进路径

演进过程中应坚持“能用程序验证的约束，不只交给模型判断”。模型适合处理偏好理解、方案权衡和自然语言表达；预算求和、日期冲突、距离计算和字段校验更适合确定性程序。

# 8 课程总结

## 8.1 工程认知转变

本项目最重要的收获是认识到 Agentic AI 开发不等同于“在 Web 页面中调用大模型”。如果只有一次提示词请求，系统对信息是否完整、事实是否可靠、调用是否失败都缺乏控制。真正可用的 Agent 需要显式状态、动作选择、外部工具、约束校验、异常处理和反馈闭环。

结构化旅行画像使自然语言需求变成可检查的数据；缺失判断让系统能够主动追问；工具层给模型提供事实上下文；本地规则规划器保证外部能力失效时仍有输出。这些传统软件工程方法并没有因为使用大模型而过时，反而成为控制生成不确定性的关键。

## 8.2 开发过程反思

项目开发过程中出现的几个问题具有代表性。“两个人”和“预算 20000”的解析错误说明自然语言边界必须通过真实样例和回归测试不断补齐；API 平台已计费但页面降级说明仅查看最终页面不足以定位分布式调用问题，需要日志和流式协议；API Key 误入示例模板的风险说明 AI 项目同样必须执行严格的配置隔离。

这些问题推动系统从“能运行的演示”逐步变为“可诊断、可测试、可降级的 MVP”。相比单纯追求更长、更漂亮的模型输出，建立稳定的软件边界更有工程价值。

## 8.3 AI IDE 使用体会

AI IDE 能快速生成基础结构、解释错误和补充测试，但开发者仍需要明确需求边界、检查代码是否与实际行为一致，并对外部调用成本、密钥安全和事实可靠性负责。高质量协作方式是提供可验证目标，让 AI 执行实现和测试，再依据真实运行结果迭代，而不是把未经验证的生成内容直接作为最终系统。

## 8.4 对课程的建议

建议课程在讲解提示词和模型能力之外，进一步增加以下实践内容：

- Agent 状态机与 workflow 设计；
- 工具调用的超时、重试、缓存和限流；
- LLM 输出的结构化验证与自动评估；
- Prompt Injection、密钥管理和数据隐私；
- Token 成本、延迟和质量之间的工程权衡；
- 从 Demo 到可部署系统的可观测性与测试方法。

这些内容能够帮助学生从“会调用模型”进一步提升到“能交付可靠的 AI 软件系统”。

## 9 附录 A：运行与复现

### 9.1 环境安装

```
uv venv
uv pip install -e '[dev]'
```

### 9.2 配置模型

```
cp .env.example .env
```

在 .env 中填写 OpenAI 兼容服务信息。真实 Key 不应出现在报告、截图或 Git 仓库中。

### 9.3 启动系统

```
uv run uvicorn src.main:app --reload
```

访问：

- Web 页面: <http://127.0.0.1:8000>
- Swagger API 文档: <http://127.0.0.1:8000/docs>
- 健康检查: <http://127.0.0.1:8000/api/health>

### 9.4 执行测试

```
uv run pytest -q
```